
JPA: Java Persistence API

Assane SECK
Ingénieur-Informaticien

JPA: Historique

- ▶ Au début des années 2000, constatant l'échec des EJB Entity, une nouvelle API voit le jour : JDO (Java Data Object). L'API est publique, mais sans implémentation de référence gratuite. Plusieurs produits commerciaux permettent de l'utiliser, citons ici le français LiDo, et l'américain Kodo. JDO existe toujours (une version 3.0 est annoncée), mais ne fait pas partie du package JEE.
- ▶ C'est également à cette époque qu'Hibernate apparaît, indépendamment de JDO et de tout processus de standardisation Java. Le fait qu'il soit le seul produit open source et gratuit de qualité industrielle sur le marché du mapping objet / relationnel, lui assure immédiatement un succès immense, et en fait en quelques années un standard de fait.

JPA: Historique

- ▶ Devant ce succès, Sun décide de créer un nouveau standard JEE : JPA (Java Persistent API). JPA aura pour implémentation de référence une version gratuite de Toplink : Toplink Essentials. Oracle donne les fonctions de base de Toplink à la fondation Apache, tout en en conservant certaines parties, vendues sous licence payante.
- ▶ Rapidement, Oracle finira par donner l'ensemble de Toplink à la fondation Eclipse, sous le nom EclipseLink. Il s'agit toujours de l'implémentation de référence de JPA.
- ▶ Bien qu'étant à l'origine du standard JPA, Hibernate n'en est pas l'implémentation de référence. Hibernate propose le support de JPA en v1 et v2, et reste massivement utilisé aujourd'hui.

JPA: Java Persistence API

- ▶ Apparue avec java 5, JPA est un Standard et partie intégrante de la plate-forme Java EE
- ▶ Automatisation
 - ▶ CRUD
 - ▶ SQL, JPQL
- ▶ Une spécification qui définit un ensemble de règles permettant la gestion du Mapping Objet java et Base de données
- ▶ La correspondance entre des objets d'une application et les tables d'une base de données est nommée ORM (Object-Relational Mapping).
- ▶ JPA a tenu compte du succès d'Hibernate , JPA est une spécification

JPA: Java Persistence API

- ▶ La persistance recouvre 3 zones essentielles :
 - ▶ l'API définie dans le paquetage javax.persistence
 - ▶ le langage JPQL (Java Persistence Query)
 - ▶ l'objet/les métadonnées relationnelles
- ▶ 3 implémentations existantes de JPA
 - ▶ EclipseLink, l'implémentation de référence ;
 - ▶ Hibernate, projet porté par JBoss, appartenant à RedHat ;
 - ▶ OpenJPA, projet de la fondation Apache.

Objet du mapping objet / relationnel

- ▶ Traditionnellement, Ecrire du code SQL dans du code Java, et à le passer à la base via un pilote JDBC. Cette méthode fonctionne correctement et est raisonnablement efficace. Elle comporte deux inconvénients majeurs :
 - ▶ Elle impose d'écrire du code SQL propre à un serveur de base de données dans du code Java ;
 - ▶ Elle impose l'écriture de code SQL répétitif, à faible valeur ajoutée, notamment toutes les opérations CRUD de base.
- ▶ Le mapping objet / relationnel vise en premier lieu à s'affranchir de ces deux contraintes.
- ▶ Cette technique consiste à associer de façon déclarative, un modèle d'objets Java (ou exprimé dans un autre langage), à une structure de tables SQL, déjà existante ou non. Cette association peut se faire de différentes façons : soit par annotation du code Java, soit en utilisant des descripteurs XML.

Objet du mapping objet / relationnel

- ▶ Une fois cette association déclarée, l'API JPA est capable de vérifier si la structure de tables existe bien en base de données, et si elle n'existe pas, elle est capable de la créer. Elle est également capable d'écrire l'ensemble des requêtes SQL nécessaires à la manipulation complète en base des objets Java instance des classes annotées.
- ▶ De plus, la façon dont les classes Java sont annotées est indépendante de la base de données utilisée. Il est donc possible d'associer un jeu de classes annotées à (à peu près) n'importe quelle base de données, sans modification du code Java.

Les Implémentations JPA

Nom Implémentation JPA	Configuration du provider
Hibernate (RedHat)	<code>org.hibernate.jpa.HibernatePersistenceProvider</code>
Eclipse Link	<code>org.eclipse.persistence.jpa.PersistenceProvider</code>
Open JPA	<code>org.apache.openjpa.persistence.PersistenceProviderImpl</code>

Annotation JPA

Annotation	Rôle
@Table	Préciser le nom de la table concernée par le mapping
@Column	Associer un champ de la table à la propriété (à utiliser sur un getter)
@Id	Associer un champ de la table à la propriété en tant que clé primaire (à utiliser sur un getter)
@GeneratedValue	Demander la génération automatique de la clé primaire au besoin
@Basic	Représenter la forme de mapping la plus simple. Cette annotation est utilisée par défaut
@Transient	Demander de ne pas tenir compte du champ lors du mapping

Annotation JPA

Attributs	Rôle
name	Nom de la table
catalog	Catalogue de la table
schema	Schéma de la table
uniqueConstraints	Contraintes d'unicité sur une ou plusieurs colonnes

Annotation JPA

Attributs	Rôle
name	Nom de la colonne
table	Nom de la table dans le cas d'un mapping multi-table
unique	Indique si la colonne est unique
nullable	Indique si la colonne est nullable
insertable	Indique si la colonne doit être prise en compte dans les requêtes de type insert
updatable	Indique si la colonne doit être prise en compte dans les requêtes de type update
columnDefinition	Précise le DDL de définition de la colonne
length	Indique la taille d'une colonne de type chaîne de caractères
precision	Indique la taille d'une colonne de type numérique
scale	Indique la précision d'une colonne de type numérique

Annotation JPA

Attributs	Rôle
strategy	Précise le type de générateur à utiliser : TABLE, SEQUENCE, IDENTITY ou AUTO. La valeur par défaut est AUTO
generator	Nom du générateur à utiliser

Un premier exemple

- ▶ Une application JPA a besoin d'un certain nombre d'éléments techniques pour fonctionner. Ces éléments sont :
 - ▶ Des classes dites "persistantes". Ce sont des classes Java classiques, qui possèdent des annotations particulières.
 - ▶ Un fichier de description persistence.xml, qui contient certaines informations dont l'implémentation JPA a besoin pour fonctionner.
 - ▶ Une base de données, dans laquelle JPA va créer des tables et enregistrer des données.
- ▶ Le but de cette première partie est de créer une première classe persistante et de la faire fonctionner. Cela va nous permettre de passer en revue l'ensemble de ces éléments techniques, et de les mettre en place.

Un Modèle Objet très simple

```
public class Musician {  
  
    private Long id;  
  
    private String name;  
  
    private Date dateOfBirth;  
  
    public MusicType musicType;  
  
}
```

Un Modèle Objet simple: Entité

```
@Entity
public class Musician {

    @Id
    private Long id;

    private String name;

    private Date dateOfBirth;

    public MusicType musicType;

}
```

Un Modèle Objet simple: Entité

```
@Entity
public class Musician {

    @Id
    private Long id;

    private String name;

    private Date dateOfBirth;

    public MusicType musicType;

}
```

Un Modèle Objet simple: Entité

```
@Entity
public class Musician {

    @Id
    private Long id;

    private String name;

    private Date dateOfBirth;

    public MusicType musicType;

}
```

Un Modèle Objet simple: Entité

Clé primaire: Stratégie de génération

```
@Entity
public class Musician {

    @Id @GeneratedValue (strategy = GenerationType.AUTO)
    private Long id;

    private String name;

    private Date dateOfBirth;

    @Enumerated(EnumType.STRING)
    public MusicType musicType;

}
```

Un Modèle Objet simple: Entité

Cas particuliers des dates:

DATE, TIME, DATETIME

@Entity

```
public class Musician {
```

```
@Id @GeneratedValue (strategy = GenerationType.IDENTITY)
```

```
private Long id;
```

```
private String name;
```

```
@Temporal(TemporalType.TIMESTAMP)
```

```
private Date dateOfBirth;
```

```
public MusicType musicType;
```

```
}
```

Un Modèle Objet simple: Entité Métadonnées

```
@Entity
@Table(name=« musiciens»)
public class Musician {

    @Id @GeneratedValue (strategy = GenerationType.AUTO)
    private Long id;
    @Column (name="name", length=80)
    private String name;

    @Temporal(TemporalType.TIMESTAMP)
    private Date dateOfBirth;

    public MusicType musicType;

}
```

Un Modèle Objet simple: Entité

Cas particuliers des énumérations

@Entity

```
public class Musician {
```

```
@Id @GeneratedValue (strategy = GenerationType.AUTO)
```

```
private Long id;
```

```
@Column (name="name", length=80)
```

```
private String name;
```

```
@Temporal(TemporalType.TIMESTAMP)
```

```
private Date dateOfBirth;
```

```
public MusicType musicType;
```

```
}
```

```
public enum MusicType {
```

```
    ROCK, CLASSIC, JAZZ
```

```
}
```

JDBC: Paramètres BD

```
private String user = "root";  
private String password = "";  
private String url = "jdbc:mysql://localhost:3306/db_school";  
private Connection conect;
```

JPA: Paramètres BD

```
spring.datasource.url = jdbc:mysql://127.0.0.1:3306/db_esp  
spring.datasource.username=root  
spring.datasource.password=
```

JDBC: Connexion BD

```
public void connectionDB() {  
  
    try {  
  
        Class.forName("com.mysql.jdbc.Driver");  
        conect = DriverManager.getConnection(url, user, password);  
        System.out.println("Connexion réussie");  
    } catch (Exception e) {  
        e.printStackTrace();  
        System.out.println("Connexion échouée");  
    }  
}
```

JPA: Connexion BD

```
spring.datasource.driverClassName = com.mysql.cj.jdbc.Driver  
spring.jpa.database=MYSQL
```

JDBC: Recherche par Id

```
public Student getOneStudent(Long id) {  
  
    Student student =new Student();  
  
    try {  
  
        connectionDB();  
        String sql = "SELECT * FROM user WHERE id=?";  
        PreparedStatement st = connect.prepareStatement(sql);  
        st.setLong(1, id);  
        ResultSet rs = st.executeQuery();  
  
        student.setId(Long.parseLong(rs.getString("id")));  
        student.setFirstName(rs.getString("first_name"));  
        student.setLastName(rs.getString("last_name"));  
        student.setGender(rs.getString("gender"));  
        student.setDate(new Date(rs.getString("date")));  
  
    } catch (Exception e) {  
        e.printStackTrace();  
    }  
    return student;  
}
```

JPA: Recherche par Id

```
return studentRepository.findById(id);
```

JDBC: recherche All

```
public List<Student> getAllStudent() {
    List<Student> list= new ArrayList<>();
    try {

        String sql = "SELECT * FROM student;";
        connectionDB();
        PreparedStatement stm = connect.prepareStatement(sql);;
        ResultSet rs = stm.executeQuery();

        while (rs.next()) {
            Student student = new Student();
            student.setId(rs.getLong("id"));
            student.setFirstName(rs.getString("first_name"));
            student.setLastName(rs.getString("last_name"));
            student.setGender(rs.getString("gender"));
            student.setDate(rs.getDate("date"));
            //rechercher la formation et setter ça
            Formation formation=null;
            student.setFormation(formation);
            list.add(student);
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return list;
}
```

JPA: recherche All

```
return studentRepository.findAll();
```

JDBC: Insertion

```
public Student saveStudent(Student student) {
    PreparedStatement stm ;
    try {
        connectionDB();
        String sql = "INSERT INTO student(first_name, last_name, gender, date, id_formation) VALUES (?, ?, ?, ?, ?)";
        stm = connect.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS);
        stm.setString(1, student.getFirstName());
        stm.setString(2, student.getLastName());
        stm.setString(3, student.getGender());
        stm.setDate(4, student.getDate());
        stm.setLong(5, student.getFormation().getId());
        stm.executeUpdate();

    } catch (Exception e) {
        e.printStackTrace();
    }

    if(stm!=null) {
        ResultSet rs = stm.getGeneratedKeys();
        student.setId(rs.getLong(0));
    }
    return student;
}
```

JPA: Insertion

```
return studentRepository.save(student);
```

JDBC: Update

```
public Student updateStudent(Student student) {  
    try {  
        connectionDB();  
        String sql = "UPDATE student SET first_name=?, last_name=?, gender=?, date=?, formation_id=? WHERE id=?";  
        PreparedStatement stm = connect.prepareStatement(sql);  
  
        stm.setString(1, student.getFirstName());  
        stm.setString(2, student.getLastName());  
        stm.setString(3, student.getGender());  
        stm.setDate(4, student.getDate());  
        stm.setLong(5, student.getFormation().getId());  
        stm.setLong(6, student.getId());  
        stm.executeUpdate();  
  
    } catch (Exception e) {  
        e.printStackTrace();  
    }  
  
    return student;  
}
```



JPA: Update

```
return studentRepository.save(student);
```

JPA: Delete

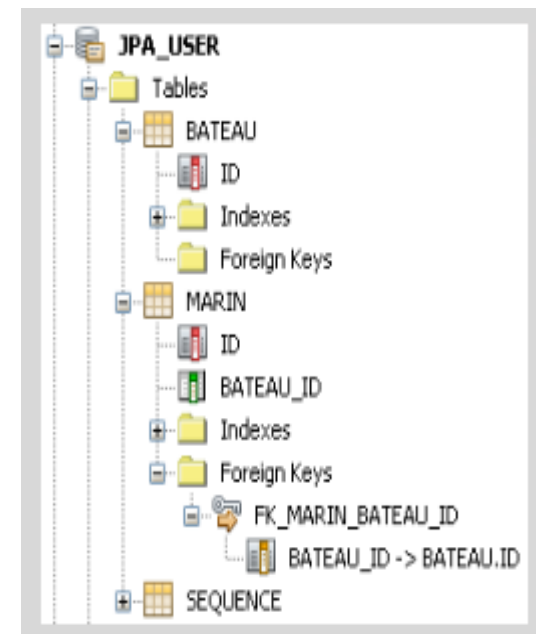
```
public boolean deleteStudent(int id) {  
    try {  
        connectionDB();  
        String sql = "DELETE FROM student WHERE id=?";  
        PreparedStatement stm = conect.prepareStatement(sql);  
        stm.setInt(1, id);  
        stm.executeUpdate();  
        return true;  
    } catch (Exception e) {  
        e.printStackTrace();  
        return false;  
    }  
}
```

JPA: Delete

```
studentRepository.delete(student);
```

ManyToOne OneToMany

```
//  
// Entité Marin  
//  
  
@Entity  
public class Marin implements Serializable {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.AUTO)  
    private Long id;  
  
    @ManyToOne  
    private Bateau bateau ;  
  
    // reste de la classe  
}  
  
//  
// Entité Bateau  
//  
@Entity  
public class Bateau implements Serializable {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.AUTO)  
    private Long id;  
  
    @OneToMany(mappedBy="bateau")  
    private Collection<Marin> marins ;  
  
    // reste de la classe  
}
```



ManyToOne OneToMany

Pour les relations, faudra créer les méthodes getter setter

Formation

@OneToMany(mappedBy = "formation")

private Set<Student> students;

Student

@ManyToOne

private Formation formation;

FIN

